UNIVERSITY PARTNER

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Artificial Intelligence and Machine Learning (6CS012)

Image Classification with Convolutional Neural Networks and Transfer Learning

Student ID: 2431895

Student Name: Saval Kaini

Section: L6CG4

Module Leader: Mr. Bijaya Ghimire

Submitted on: 05/10/2026

Abstract

This report deals with the detailed analysis of image classification technique for the Flowers Recognition Dataset which contains 4317 images of 5 categories of flowers using Convolutional Neural Networks (CNNs). The experiment has been split into two components.The experiment is carried out in two parts. In part A, different architectures for the CNNs from scratch have been considered, as well as a comparison experiment with the baseline model and the deeper model and with other optimization methods and regularization methods. The experiments which were run for the pre-trained CNN models with transfer learning for MobileNetV2 and VGG16 models are described in Part B. From the results obtained, the performance of the baseline CNN was found to be at 78% and the deeper one had a score of 73% in the validation phase. It was observed that the top two models i.e., the transfer learning models were the best with the MobileNetV2 giving 92.76% on validation and VGG16 giving 90.14% on validation. Simple models seem to perform better on low data sets than on big data sets and Adam appears to converge faster than SGD. Transfer learning can solve the problems of small data volume. The experiments were run on T4GPUs.

Table of Contents

1 Introduction 1

2 Dataset 2

2.1 Dataset Description 2

2.2 Dataset Statistics 2

2.3 Dataset Split 3

2.4 Preprocessing 3

2.5 Data Augmentation 4

3 Methodology 5

3.1 Part A: CNN From Scratch 5

3.2 Baseline CNN Model 5

3.3 Deeper CNN with Regularization 5

3.4 Training Configuration 6

3.5 Part B: Transfer Learning 6

3.6 MobileNetV2 6

3.7 VGG16 6

4 Experiments and Results 7

4.1 Baseline vs Deeper Model Performance 7

4.2 Computational Efficiency 8

4.3 Optimizer Analysis: SGD vs Adam 8

4.4 Ablation Study: Dropout Removal 9

4.5 Transfer Learning Results 10

5 Overall Comparison 12

6 Conclusion and Future Work 13

6.1 Key Findings 13

6.2 Limitations 14

6.3 Future Work 15

6.4 Hardware 16

7 Appendix 16

1 Introduction

The image classification is one of the most widely occurring problems in computer vision and has been used in various fields such as medicine, self-driving cars, agriculture and nature protection. Today the new summit of image classification is achieved using deep learning techniques such as Convolutional Neural Networks (CNN), which learn to represent features hierarchically, directly from pixel values, without any handcrafted features.

The recognition problem of the flowers is difficult and challenging because the flowers can be confused with one another and a host of changes in the background, illumination, orientation and structure occur between flowers.

The goal of this work is to explore the possibility of using the CNNs in the flower's recognition problem either from scratch or transfer learning approach. This research explores the effect of the depth of the network, regularizations, optimizers and pretraining on the performance of CNNs. Flowers Recognition Dataset, which is available on Kaggle, was employed for the experiments with five flower classes.

2 Dataset

2.1 Dataset Description

The Flowers Recognition Dataset was taken from Kaggle and consists of real-world pictures of flowers that belong to five different categories: daisy, dandelion, rose, sunflower, and tulip. This dataset has been collected from different sources which includes various flowers with different sizes, background, orientation and lighting.

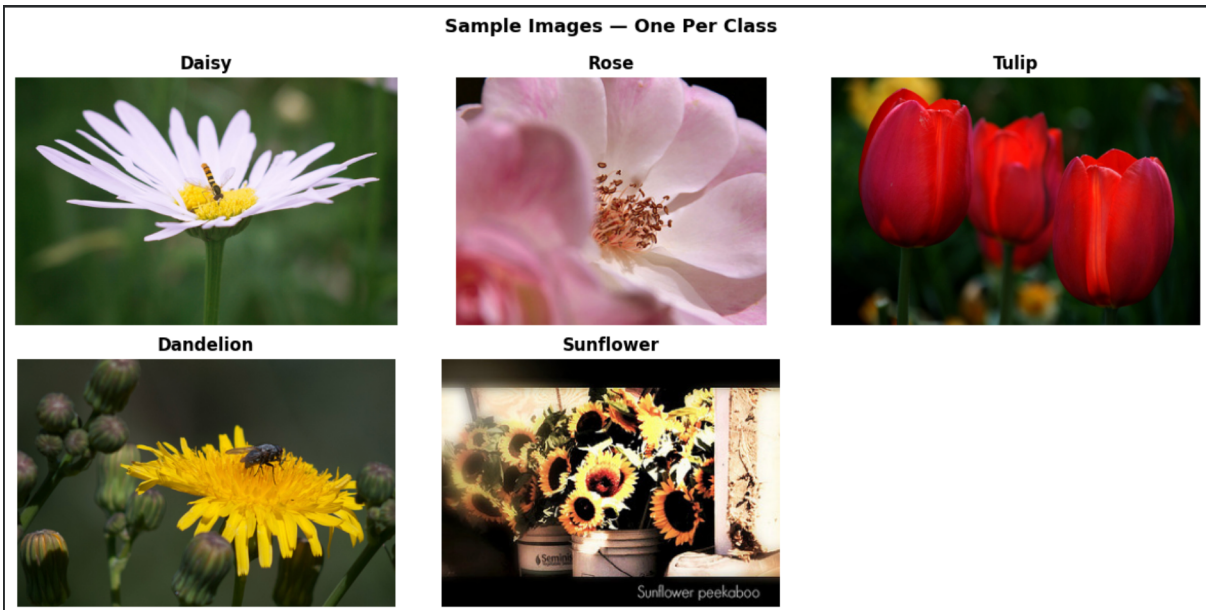


Figure 1: Sample Image from each Class

2.2 Dataset Statistics

The dataset contains a total of 4317 images distributed among five different classes as follows:

- Dandelion: 1052 images
- Tulip: 984 images
- Rose: 784 images
- Daisy: 764 images
- Sunflower: 733 images

As shown in figure 2 there is mild imbalance of class on with dandelion having 43.5% more images than sunflower. Upon verification no corrupted images were found which were done using PIL image verification.

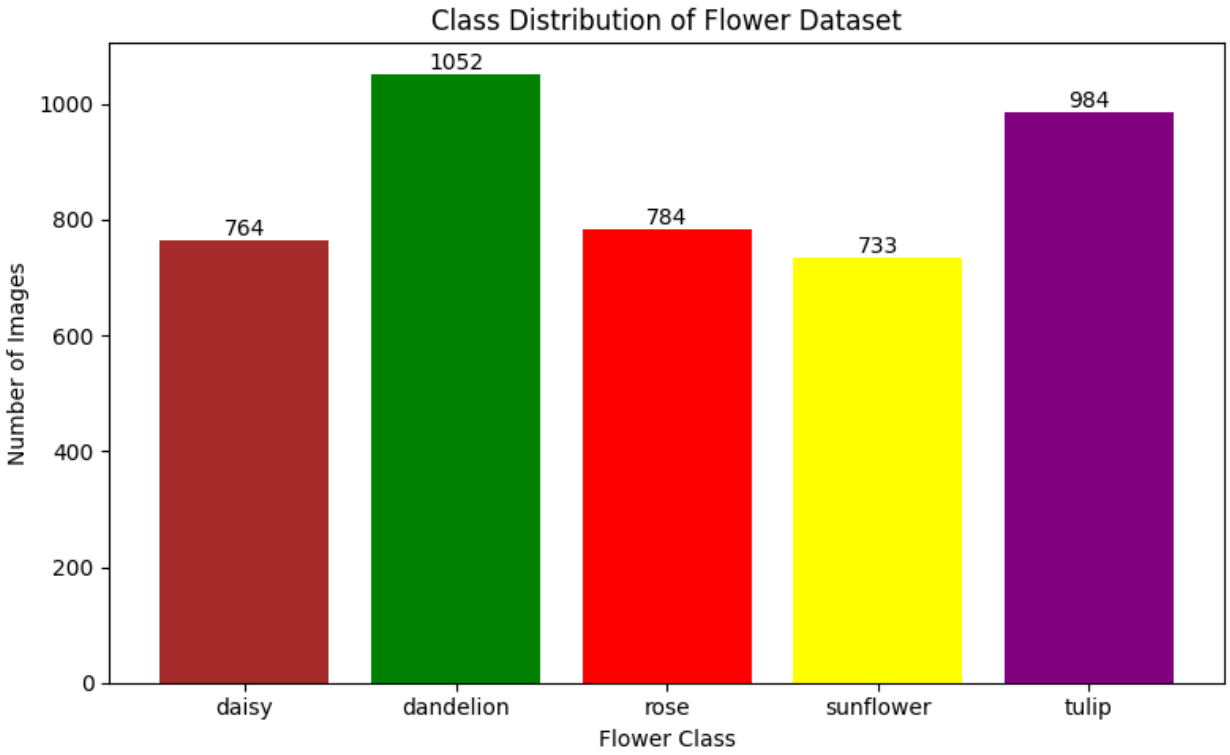


Figure 2: Class Distribution Bar Chart

2.3 Dataset Split

This data set only contains training data. Thus, the same data was again divided into the testing and the validation sets with sklearn train test split with random_state=42 to guarantee reproducibility:

⇒ Training Set: 3019 images (70%)

⇒ Validation Set: 649 images (15%)

⇒ Test Set: 649 images (15%)

The 70/15/15 split was selected to maintain a balance when training, validating and testing the data to ensure our model is learning effectively without tuning too much and for a good evaluation.

2.4 Preprocessing

The different preprocessing techniques were applied:

⇒ Resizing: All the images were resized to 224×224 to match the input dimension of CNN and to be compatible with the pretrained models used in Part B.

⇒ Normalization: To stabilize the training and to boost the model's performance, the pixel values were rescaled from [0, 255] to [0, 1] by applying rescale=1./255.

⇒ Corrupted Image Check: Before splitting the data corrupted images were checked using PIL image verification. However, no corrupted images were found.

2.5 Data Augmentation

Data augmentation of only training data was performed to enhance dataset size and generalization using KerasImageDataGenerator:

- rotation_range = 30: 30 degrees random rotation
- width_shift_range = 0.2: Horizontal shift by 20%
- height_shift_range = 0.2: vertical shift by 20%
- shear_range = 0.2: shear transformation
- zoom_range = 0.2: Zoom in or out by 20%
- horizontal_flip = True: Flip image horizontally

Validation and test generators were applied only rescaling without augmentation to ensure fair and consistent evaluation on original unmodified images.

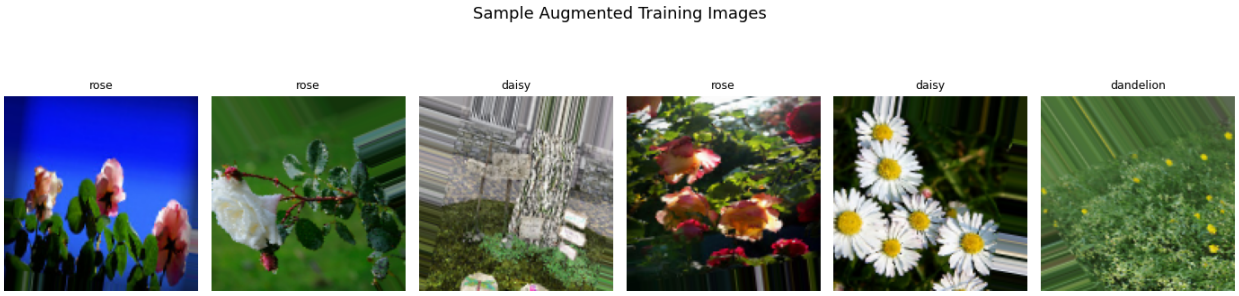


Figure 3: Sample of Agumeneted Images

3 Methodology

3.1 Part A: CNN From Scratch

3.2 Baseline CNN Model

The baseline model was designed with a simple architecture which consists of 3 convolutional block, 1 flatten layer, 3 Fully connected layer and Output layer with dense 5 and softmax activation function. The model is comprised of:

⇒ Convolutional Block 1: Conv2D (32, 3×3, ReLU) with MaxPooling2D (2×2)

⇒ Convolutional Block 2: Conv2D (64, 3×3, ReLU) with MaxPooling2D (2×2)

⇒ Convolutional Block 3: Conv2D (128, 3×3, ReLU) with MaxPooling2D (2×2)

⇒ Flatten Layer

The fully connected layer is a dense layer with 256 units using the ReLU activation function.

To cite this event: fully connected layer 2 dense (128, ReLU)

Fully Connected Layer 3: Dense (64, ReLU)

Output Layer: Dense (5, Softmax)

The kernel size was selected to be 3x3 because it is a standard size and is most effective for feature extraction. To capture increasingly complex features at each layer Filters double progressively (32 to 64 to 128). To avoid vanishing gradient ReLU activation was used.

3.3 Deeper CNN with Regularization

The deeper model has two extra convolutional layers and adds the regularization. The model consists of two Convolutional Blocks: each with 2 Conv2D layers, Batch Normalization, MaxPooling2D and Dropout (0.25)

⇒ Filters: 32 ,32, 64, 64, 128, 128, 256, 256

⇒ Fully Connected Layers: Dense (512) to Dense(256) to Dense(128) with BatchNormalization and Dropout of (0.5/0.3)

⇒ Input Layer: Dense (5, Sigmoid) Output Layer: Dense (5, Softmax)

After each Conv layer, the same padding was used in order to keep the size of the spatial dimensions. To help train the model, activations are normalized after every convolution layer using BatchNormalization. During training, the dropout randomly discards the neurons in order to avoid memorizing the training data.

3.4 Training Configuration

All models were trained with the following configuration:

- Optimizer: Adam (learning_rate=0.001) for baseline and deeper models
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy
- Epochs: 20 maximum
- Batch Size: 32
- Callbacks: EarlyStopping(patience = 5, restore_best_weights = True) and ReduceLROnPlateu(factor=0.5, patience = 3, lr = 1e-7)

3.5 Part B: Transfer Learning

3.6 MobileNetV2

MobileNetV2 was used as the main transfer learning model because of its low architecture (3.4M parameters), being efficient when working with small datasets, and being able to take inputs of 224×224. The training was conducted in two steps:

Phase 1: Feature Extraction (10 epochs), all base layers of the MobileNetV2 frozen. Only newly added classification head trained with learning_rate=0.001.

• Phase 2 - Fine Tuning (10 epochs): Top 30 layers unfrozen. Catastrophic forgetting was prevented by training the entire model using lower learning_rate=0.0001.

New classification head: GlobalAveragePooling2D, Dense(256, ReLU), BatchNormalization, Dropout(0.3) Dense(128, ReLU), Dropout(0.2), Dense(5, Softmax))

3.7 VGG16

VGG16 was chosen as secondary transfer learning model for comparison. It is a more powerful architecture than MobileNetV2 with 138M parameters,

⇒ Phase 1 - Feature Extraction (10 epochs): All the base layers of VGG16 are frozen. This is a classification head, which has been trained using learning_rate=0.001.

⇒ Phase 2 - Fine Tuning (10 epochs): Top 4 layers unfrozen. Model trained using learning_rate=0.0001. Early stopping has been triggered at epoch 9, best weights have been restored from epoch 4.

Same classification head architecture as MobileNetV2 was used for fair comparison.

4 Experiments and Results

4.1 Baseline vs Deeper Model Performance

Model	Val Accuracy	Val Loss	Training Time (in secs)
Baseline CNN	78%	0.6128	759.99
Deeper CNN (Adam)	73%	0.7112	877.27

Even though the basic model hit 78% on validation, outdoing the deeper one at 73%, that gap tells a story about fit - how deep networks need more data than they get here. With just over three thousand examples to train on, the larger setup struggled, stumbling through learning instead of flowing. Meanwhile, the simpler version moved steadily forward, showing no stress, its training score dipping only slightly below validation by less than a quarter point. That narrow space between practice and test suggests balance, not strain.

Deep inside the network, things started rough - validation error hovered near 3.0 right at the beginning. Though it dipped over time, the path wasn't smooth, thanks to Dropout shaking up each step. That same Dropout kept memorization in check, yet made settling on a final answer harder because so little data was fed into training.

Surprisingly low numbers pop up for rose across both setups - F1 lands at 0.70 in the simpler version, drops to 0.52 when things go deeper. Wild shifts in color and form likely trip up recognition every time. Meanwhile, sunflowers stand out sharply, topping charts regardless of

structure. Their look just makes sorting them easier somehow.

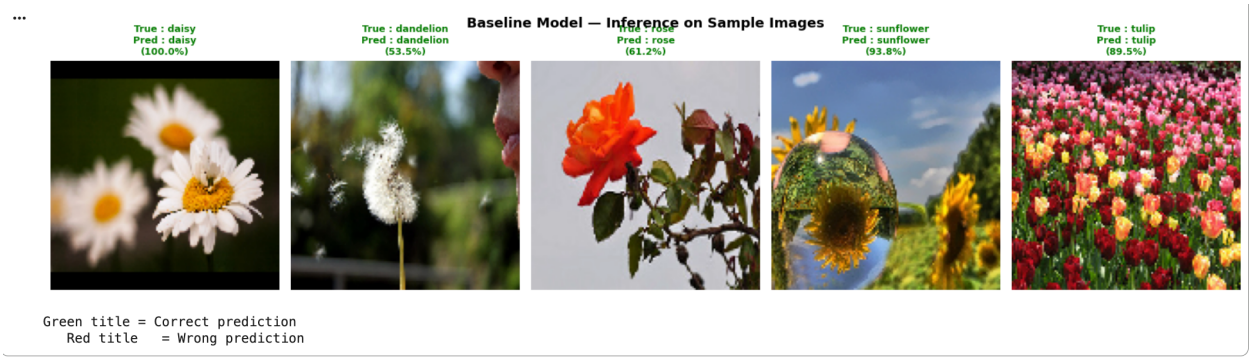


Figure 4: Bassline Model Sample Predictions

4.2 Computational Efficiency

The base model finished training in just under 760 seconds. Training the deeper version? That ran for 877 seconds - roughly a fifteen percent hike. Even with heavier computation, its guesses were less on target. So when you weigh speed against correctness per data point, it didn't come out ahead here. Give it far more pictures though - say, over ten thousand per category - and that balance might flip entirely.

4.3 Optimizer Analysis: SGD vs Adam

Model	Val Accuracy	Val Loss	Training Time (in secs)
Deeper CNN (Adam)	73%	0.7112	877.27
Deeper CNN (SGD)	61%	0.9754	874.03

Though it started slower, Adam reached 73% accuracy, up from SGD's 61%, showing a 12-point gain over the validation phase. Thanks to stored gradient info, each parameter gets custom step sizes during updates - this shapes how Adam handles bumpy error surfaces. SGD pushed ahead early, racing forward when locked at 0.01 learning speed and boosted by 0.9 momentum. By epoch 19, models using steady rates crashed - from 67% down to just 37% correct guesses - proof of shaky progress. Because of these swings, picking Adam makes more sense when tuning convolution networks on limited

image groups.

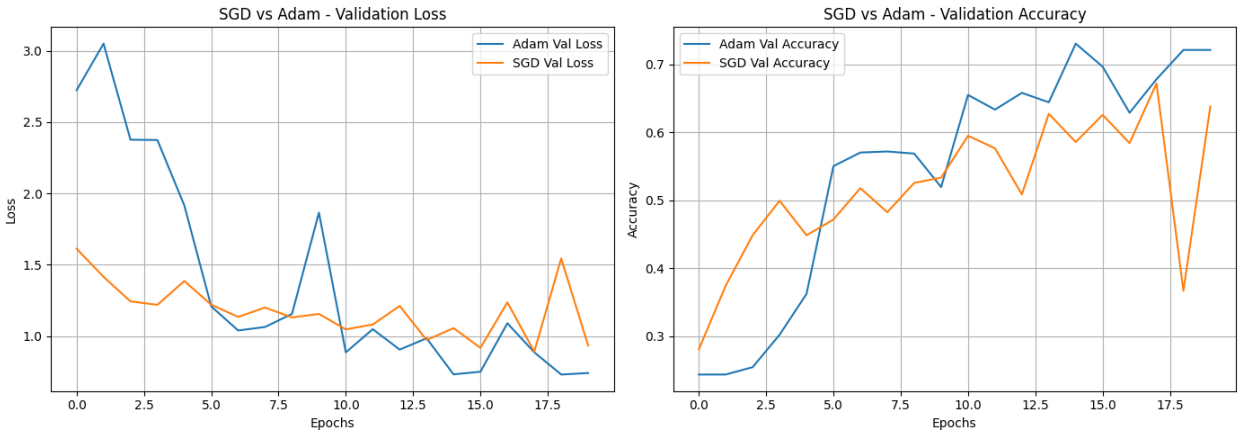


Figure 5: SGD VS Adam Validation loss and Accuracy comparison

4.4 Ablation Study: Dropout Removal

Model	Val Accuracy	Val Loss
Deeper CNN with Dropout	73%	0.7112
Deeper CNN without Dropout	81%	0.5354

Surprisingly, taking out Dropout made the deeper model perform better on validation, jumping from 73% to 81%. By the fifth epoch, the version without Dropout already hit 65% accuracy, while the one with it managed only 52%. Faster progress appeared early in training when Dropout was absent. Although Dropout often helps prevent overfitting, here its removal didn't cause major issues - training and validation errors stayed close, differing by just around 1%.

Looking at the outcome, a dropout rate of 0.25 or 0.5 ended up being excessive when working with flower images in small datasets - it caused poor learning instead of preventing it. Surprisingly, BatchNormalization on its own handled regularization well enough here. For even tinier datasets, better results came from dropping just 10% to 15%, though skipping dropout altogether sometimes

worked fine too.

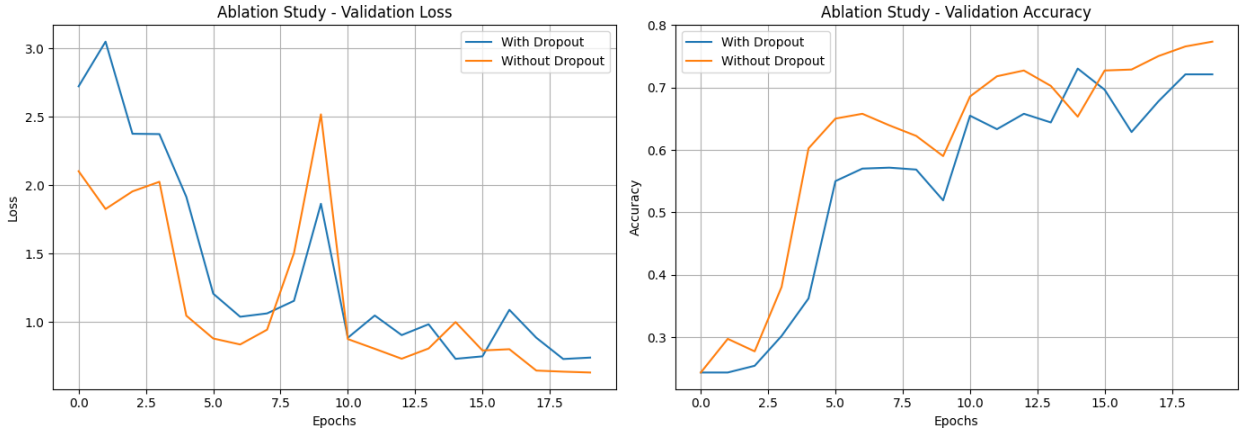


Figure 6: Validation Plot

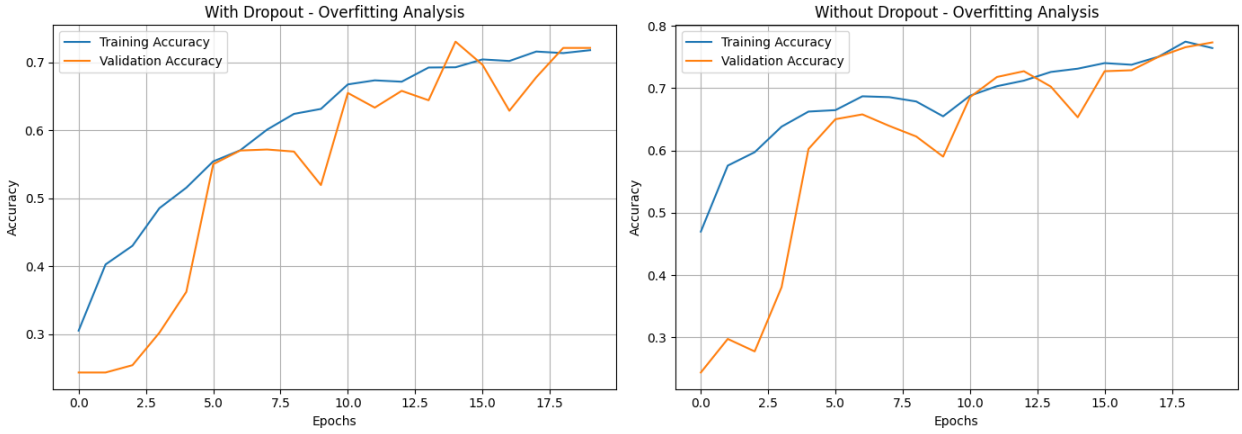


Figure 7: Analysis Plot

4.5 Transfer Learning Results

Model	Val Accuracy	Val Loss	Training Time (in sec)
MobileNetV2	92.76%	0.2894	841.76
VGG16	90.14%	0.2986	820.28

⇒ One step ahead, the two transfer learning models outperformed every model trained from scratch. Reaching nearly 93%, MobileNetV2 scored highest during validation, while VGG16 landed just under 90.5%. Look at how consistently strong MobileNetV2

performs across each category - evidence sits in the image that follows.

Classification Report - MobileNetV2:

	precision	recall	f1-score	support
daisy	0.89	0.91	0.90	115
dandelion	0.98	0.91	0.94	158
rose	0.89	0.93	0.91	118
sunflower	0.91	0.95	0.93	110
tulip	0.94	0.91	0.92	148
accuracy			0.92	649
macro avg	0.92	0.92	0.92	649
weighted avg	0.92	0.92	0.92	649

Figure 8: Classification Report – MobileNetV2

Among all classes, roses stood out with a jump from F1=0.70 to F1=0.92 using MobileNetV2 - gaining 0.22 points over the baseline. Pretrained features from ImageNet clearly picked up on subtle rose details that simpler models missed due to scarce training examples. While VGG16 did well too, it trailed just behind MobileNetV2, as seen in the plot underneath.

Classification Report - VGG16:

	precision	recall	f1-score	support
daisy	0.91	0.93	0.92	115
dandelion	0.95	0.91	0.93	158
rose	0.78	0.95	0.86	118
sunflower	0.94	0.94	0.94	110
tulip	0.94	0.81	0.87	148
accuracy			0.90	649
macro avg	0.90	0.91	0.90	649
weighted avg	0.91	0.90	0.90	649

Figure 9: Classification Report VGG16

By the ninth round, VGG16 had already stopped early - its progress paused during recovery after hitting the fourth round - while MobileNetV2 didn't restore its state until much later, near round twenty-two. Even though VGG16 carried far more numbers under

the hood - one hundred thirty-eight million compared to just three point four million - its performance lagged behind in precision, revealing a twist: leaner designs tend to adapt faster on limited data when adjusted carefully.

Right off, MobileNetV2 pulled out useful patterns early - validation hit 84% by the first round using ImageNet's prior learning. That alone shows how well those prebuilt features adapt to telling flowers apart. Then came adjustments tuned just for petals and shapes. Performance climbed again, landing near 93%. Turned out, tweaking the model made it far sharper at spotting bloom details.

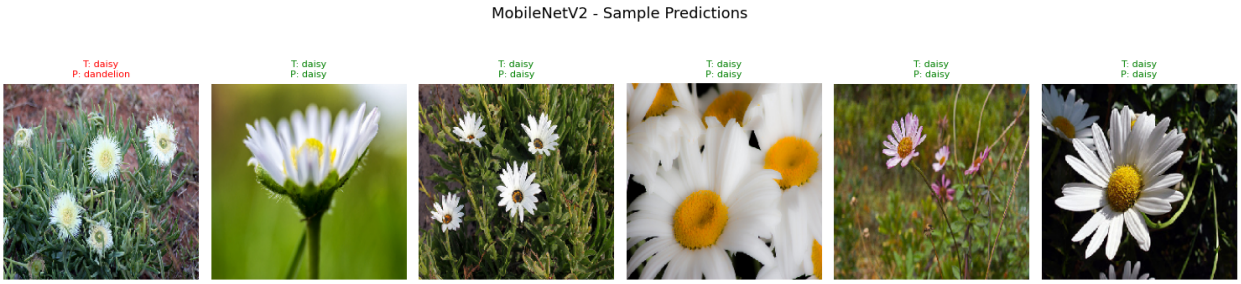


Figure 10: Sample Predections MobileNetV2

5 Overall Comparison

Below is a list of how each model did during testing, based on how accurate they were, their error rates, along with how long it took them to train.

Surprisingly, MobileNetV2 took the lead - hitting 92.76% accuracy while showing just 0.2894 in validation loss. Close behind stood VGG16, landing at 90.14%. What set them apart? Both used transfer learning, which clearly edged past every model built from nothing. The gap made it obvious: starting fresh didn’t pay off.

Best result came from the deeper CNN without dropout: 81.36%. That's a jump of 3.39 percentage points above the baseline CNN at 77.97%. Small data? Batch normalization

alone handles it well. Simplicity stood out - baseline CNN ranked fourth despite minimal design. Its performance matched more complex setups, landing exactly at 77.97%.

At position five, a deeper CNN using Adam reached 73.04% accuracy. Training the same model with SGD ended at 60.86%, lowest among results. Its validation loss peaked at 0.9754. This run showed clear performance drop compared to others.

Surprisingly fast, MobileNetV2 hits 92.76% accuracy in just 841.76 seconds. That speed stands out when lined up against the deeper model - no dropout included - which crawls past at 890.74 seconds while only reaching 81.36%. So even though both aim high, one clearly moves quicker. Time spent matters as much as the result. Efficiency shows where gains truly land.

Clear patterns stand out. Transfer learning works far better than starting fresh, especially with tiny datasets - when data lacks size, it shows. Bigger models? They flop without solid training. Complexity alone won't lift performance if the foundation wobbles.

Model	Val Accuracy	Val Loss	Time(s)
Baseline CNN	0.7797	0.6128	759.99
Deeper CNN (Adam)	0.7304	0.7112	877.27
Deeper CNN (SGD)	0.6086	0.9754	874.03
Deeper CNN (No Dropout)	0.8136	0.5354	890.74
MobileNetV2 (Transfer)	0.9276	0.2894	841.76
VGG16 (Transfer)	0.9014	0.2986	820.28

Figure 11: Overall Comparison of model

6 Conclusion and Future Work

6.1 Key Findings

This research shows some significant results for deep learning-based image classification:

Out of nowhere, MobileNetV2 hit 92.76% accuracy thanks to transfer learning. Training models without pre-training lagged behind sharply - top result was just 81.36%. That gap? Eleven point four percent wider than scratch methods. Pre-trained ImageNet features quietly

carry useful patterns into narrow tasks like telling flowers apart. Especially when there isn't much data around for training.

Smaller data means a leaner model works better. When tested on just 3019 images, the eight-layer CNN didn't beat the three-layer version. Too many layers brought extra complexity that couldn't stabilize during learning. Accuracy dipped while processing demands climbed. Heavy models struggle when examples are few.

Surprisingly, the Adam optimizer handles CNN training more effectively with limited data. Its edge shows up clearly - accuracy jumps by 12 percent over SGD in these cases. This happens due to how it adjusts learning rates on the fly. Because of that, it navigates tricky loss surfaces without getting stuck. The reason lies in its built-in adaptability, making progress where others stall.

Too much dropout hurt performance here - sample was tiny, so cutting connections at 0.25 and 0.5 made learning worse. Accuracy dropped by eight points compared to using only batch normalization. When data is scarce, shrinking regularization helps. Adjusting that balance matters more than applying fixed rules.

Surprisingly, MobileNetV2 beats VGG16 when applying transfer learning on tiny datasets - even though it uses way less weight, just 3.4 million compared to 138 million. Because of how they're built - with depthwise separable convolutions - leaner models adapt quicker when data is limited. Efficiency shows up right where you need it: during fine-tuning with scarce examples.

6.2 Limitations

⇒ With just 4317 pictures, training advanced models from the beginning becomes a challenge.

⇒ More dandelion pictures showed up - about 43.5 percent extra compared to sunflowers - that could've nudged how well each group did. Class balance wasn't perfect, so results shifted a bit between types.

⇒ Even when you fix the seed, outcomes might shift a bit each time. That happens because weight initialization brings its own unpredictability. On top of that, how GPUs handle operations isn't always repeatable. Small differences creep in without warning.

⇒ Surprising differences within the same type made roses hardest to identify across every model tested. Despite similar setups, each system struggled most right there.

6.3 Future Work

⇒ A bigger collection of photos could help - especially for types like sunflowers, which show up too rarely. More examples mean the system learns better, simply because it sees more variety. When one kind appears just a few times, results get shaky. Filling those gaps makes patterns clearer across the board.

⇒ Maybe try ResNet50, EfficientNet, or InceptionV3 - these could boost how well the model performs. Starting elsewhere might work too.

⇒ Starting off, fine-tuning settings like how fast a model learns, the amount of random shutdown in neurons, or group size during training often leads to better results. One way this happens is through smart guessing based on past tries. Instead of brute-force testing, choices get sharper over time.

⇒ Learning speed might drop here while batch count rises there - balance matters. Through repeated trials guided by feedback, systems adjust these knobs automatically. Progress shows up quietly, without sudden leaps. Decisions build slowly, shaped by what worked before.

⇒ One way to boost performance is combining forecasts from separate systems, such as MobileNetV2 alongside VGG16. Instead of relying on just one network, blending outputs often leads to better results. When multiple models contribute, small mistakes tend to balance out across predictions. This approach uses diversity in design to improve overall reliability. Accuracy gains come from averaging decisions rather than trusting a single source.

⇒ When some groups show up less often, boosting their numbers - or adjusting how much attention they get - can lift results for those smaller categories. Not every sample has equal say; tipping the balance changes outcomes in subtle ways. Smaller clusters gain strength when treated with extra care during setup.

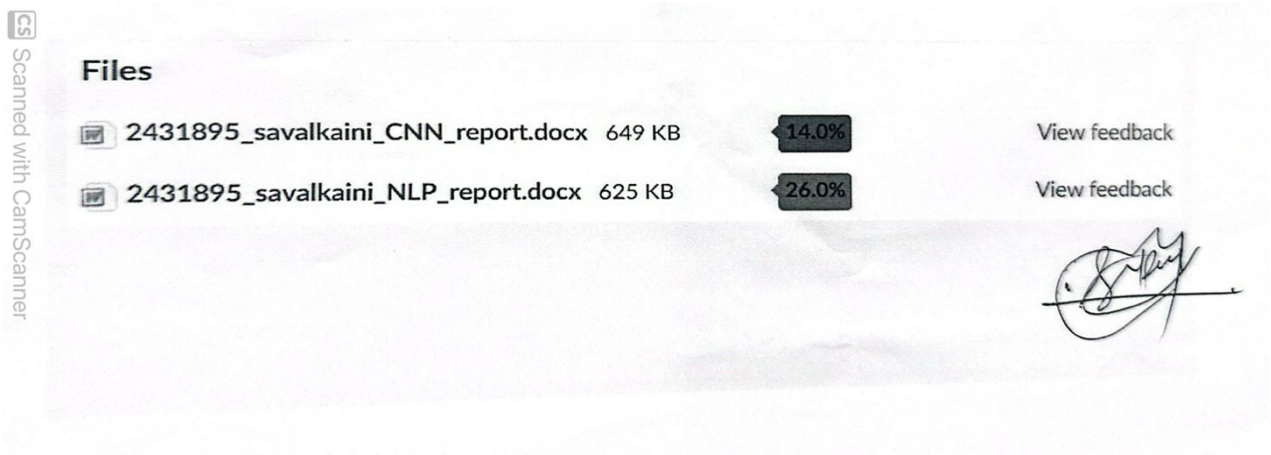
⇒ Weight shifts or copy tricks alter how models learn from rare cases. Attention tweaks nudge accuracy where it usually dips. Rare types stand out more when the system leans their way. Equal effort does not mean fair treatment across uneven ground.

6.4 Hardware

Out of nowhere, tests ran on Kaggle Notebook with two GPU accelerators. Because of the GPU, training took much less time. That made it possible to try different model setups and tweak how they were built.

7 Appendix

Signed plagiarism report by **Suren Tamang** on Viva.



Scanned with CamScanner